

Introduction:

The goal of this project is to forecast the Bitcoin market price (btc_market_price) at a 7-day horizon using longitudinal features available in the dataset. The prediction target is the log-transformed market price at time t+7 (target_log_price). Bitcoin is known for its extreme volatility, and forecasting its future prices most accurately requires careful handling of noisy, high-frequency, and longitudinal data.

The dataset contains daily Bitcoin market prices over multiple years. Since the data is inherently sequential, we cannot just randomly split into training and test sets. Therefore, we have to construct time-ordered splits and design and implement models that use information from past days to predict future prices.

Specifically, I focused on predicting the log-price of Bitcoin seven days ahead. This required

1. Reconstructing the dataset
2. Engineering lagged features such as log_price_lag1 ... log_price_lag14 and short-term rolling features (7 and 14-day moving averages and 7-day volatility).
3. Using time-ordered splits with no random shuffle to avoid leakage.
4. Applying multiple regression methods:
 - a. OLS
 - b. Best-subset selection
 - c. Ridge regression
 - d. Lasso regression

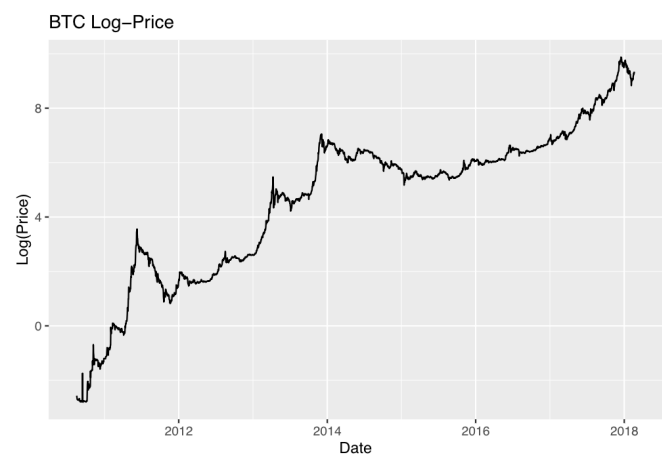
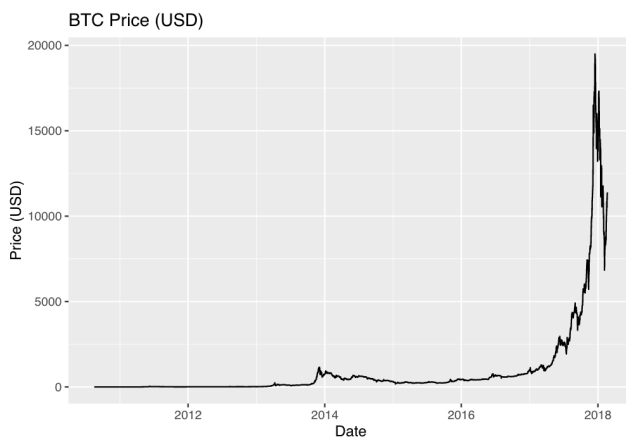
Multiple of these models needed to be tested in order to find the best performing model with lowest RMSE. Therefore, I compared all methods against a naive persistence baseline, which assumes that the price 7 days ahead will equal today's price.

```
## 'data.frame': 2920 obs. of 24 variables:
## $ Date : Date, format: "2010-02-23" "2010-02-24" ...
## $ btc_market_price : num 0 0 0 0 0 0 0 0 0 ...
## $ btc_total_bitcoins : num 2110700 2120200 2127600 2136100 2144750
## $ btc_market_cap : num 0 0 0 0 0 0 0 0 0 ...
## $ btc_trade_volume : num 0 0 0 0 0 0 0 0 0 ...
## $ btc_blocks_size : num 0 0 0 0 0 0 0 0 0 ...
## $ btc_avg_block_size : num 0.000216 0.000282 0.000227 0.000319 0.000319 0.000319 0.000319 0.000319 ...
## $ btc_n_orphaned_blocks : num 0 0 0 0 0 0 0 0 0 ...
## $ btc_n_transactions_per_block : num 1 1 1 1 1 1 1 1 1 ...
## $ btc_median_confirmation_time : num 0 0 0 0 0 0 0 0 0 ...
## $ btc_hash_rate : num 3.15e-05 3.57e-05 2.78e-05 3.20e-05 3.20e-05 3.20e-05 3.20e-05 3.20e-05 ...
## $ btc_difficulty : num 2.53 3.78 3.78 3.78 3.78 3.78 ...
## $ btc_miners_revenue : num 0 0 0 0 0 0 0 0 0 ...
## $ btc_transaction_fees : num 0 0 0 0 0 0 0 0 0 ...
## $ btc_cost_per_transaction_percent : num 25100 179.2 1057.1 64.6 1922.2 ...
## $ btc_cost_per_transaction : num 0 0 0 0 0 0 0 0 0 ...
## $ btc_n_unique_addresses : num 252 195 150 176 176 165 187 203 150 187
## $ btc_n_transactions : num 252 196 150 176 176 165 187 204 151 187
## $ btc_n_transactions_total : num 42613 42809 42959 43135 43311 ...
## $ btc_n_transactions_excluding_popular : num 252 196 150 176 176 165 187 204 151 187
## $ btc_n_transactions_excluding_chains_longer_than_100 : num 252 196 150 176 176 165 187 204 151 187
## $ btc_output_volume : num 12600 14800 8100 29349 9101 ...
## $ btc_estimated_transaction_volume : num 50 5300 700 13162 450 ...
## $ btc_estimated_transaction_volume_usd : num 0 0 0 0 0 0 0 0 0 ...
```

Data Cleaning Process and Feature Engineering:

The raw data included some missing values and zeroes in the market price variable. Therefore, to reconstruct the dataset for cleaner use, I :

1. **Dates:** Date column parsed to Date objects and data sorted chronologically.
2. **Replaced zero values with NA** (interpreted as missing): Observations where `btc_market_price <= 0` were considered missing (these are impossible values that are placeholders that will skew results) These zeros were set to NA and short gaps were linearly interpolated using `na.approx`.
3. **Explored outliers:** The proportion of price points outside IQR bounds is 11 %. However, I decided to keep them because outliers are crucial for analysis and Bitcoin can be unpredictable. Outliers were retained because they reflect real market shocks rather than measurement errors.
4. **Checked categorical predictors:** no categorical predictors were present in the dataset.
5. **Applied linear interpolation** to fill short gaps while preserving the time order.
6. **Log-transform:** $\log_price = \log(\text{btc_market_price})$ to stabilize variance.



7. Feature Engineering :

- **Constructed lag features:** `log_price_lag1 .. log_price_lag14`
- **Moving Averages*:** 7-day and 14-day rolling means (`ma7`, `ma14`)
- **Volatility:** 7-day rolling standard deviation (`vol7`).
- **Target variable:** `target_log_price = lead(log_price,7)` was constructed and rows lacking any required feature/target were dropped.

*Time series knowledge such as moving average, learned in STAT 429

Reconstructed Data:

```
## 'data.frame': 2703 obs. of 43 variables:
## $ Date : Date, format: "2010-08-31" "2010-09-01" ...
## $ btc_market_price : num 0.065 0.0649 0.0629 0.0634 0.0613 ...
## $ btc_total_bitcoins : num 3870200 3879000 3886550 3894550 3903500
## $ btc_market_cap : num 251447 251747 244464 246914 239285 ...
## $ btc_trade_volume : num 893.2 429.9 489 53.5 198.1 ...
## $ btc_blocks_size : num 1 1 1 1 1 1 1 1 1 ...
## $ btc_avg_block_size : num 0.001067 0.000832 0.000572 0.00053 0.00
## $ btc_n_orphaned_blocks : num 0 0 0 0 0 0 0 0 0 ...
## $ btc_n_transactions_per_block : num 1 1 1 1 2 2 1 1 2 2 ...
## $ btc_median_confirmation_time : num 0 0 0 0 0 0 0 0 0 ...
## $ btc_hash_rate : num 0.00542 0.00545 0.00468 0.00496 0.00555
## $ btc_difficulty : num 623 623 623 623 623 ...
## $ btc_miners_revenue : num 568 571 475 507 549 ...
## $ btc_transaction_fees : num 2.0e-01 1.3e-01 1.0e-07 0.0 0.0 ...
## $ btc_cost_per_transaction_percent : num 24.2 83 42.1 29 28.8 ...
## $ btc_cost_per_transaction : num 1.22 1.4 1.54 1.62 1.92 ...
## $ btc_n_unique_addresses : num 462 459 340 363 313 351 326 284 332 383
## $ btc_n_transactions : num 465 408 308 314 286 303 298 271 292 343
## $ btc_n_transactions_total : num 109805 110213 110521 110835 111121 ...
## $ btc_n_transactions_excluding_popular : num 465 408 308 314 286 303 298 271 292 343
## $ btc_n_transactions_excluding_chains_longer_than_100 : num 465 408 308 314 286 303 298 271 292 343
## $ btc_estimated_transaction_volume_usd : num 2349 688 1127 1751 1906 ...
## $ log_price : num -2.73 -2.73 -2.77 -2.76 -2.79 ...
## $ log_price_lag1 : num -2.67 -2.73 -2.73 -2.77 -2.76 ...
## $ log_price_lag2 : num -2.74 -2.67 -2.73 -2.73 -2.77 ...
## $ log_price_lag3 : num -2.73 -2.74 -2.67 -2.73 -2.73 ...
## $ log_price_lag4 : num -2.73 -2.73 -2.74 -2.67 -2.73 ...
## $ log_price_lag5 : num -2.71 -2.73 -2.73 -2.74 -2.67 ...
## $ log_price_lag6 : num -2.71 -2.71 -2.73 -2.73 -2.74 ...
## $ log_price_lag7 : num -2.7 -2.71 -2.71 -2.73 -2.73 ...
## $ log_price_lag8 : num -2.72 -2.7 -2.71 -2.71 -2.73 ...
## $ log_price_lag9 : num -2.71 -2.72 -2.7 -2.71 -2.71 ...
## $ log_price_lag10 : num -2.7 -2.71 -2.72 -2.7 -2.71 ...
## $ log_price_lag11 : num -2.71 -2.7 -2.71 -2.72 -2.7 ...
## $ log_price_lag12 : num -2.68 -2.71 -2.7 -2.71 -2.72 ...
## $ log_price_lag13 : num -2.6 -2.68 -2.71 -2.7 -2.71 ...
## $ log_price_lag14 : num -2.57 -2.6 -2.68 -2.71 -2.7 ...
## $ ma7 : num -2.72 -2.72 -2.73 -2.73 -2.74 ...
## $ ma14 : num -2.7 -2.71 -2.72 -2.72 -2.73 ...
## $ vol7 : num 0.0229 0.0232 0.0277 0.0297 0.0369 ...
## $ target_log_price : num -2.78 -2.78 -2.77 -2.78 -2.78 ...
```

Data contains 2703 usable rows after feature engineering.

Train/Validation/Test Split (time-aware): All splits were done chronologically to prevent look-ahead leakage. With N = 2703 usable rows after feature engineering, the following splits were used:

- **Training:** first 70% training → 1892 rows
- **Validation:** next 15 % → 405 rows
- **Test (final holdout):** final 15% → 406 rows

When building lag features and the target, appropriate rows near the start/end were dropped so every row used in modeling had a complete set of features and the 7-day-ahead target.

Methodologies:

I evaluated the following candidate predictors and regression models. All model evaluations used the validation set for hyperparameter/model-size selection and the final model was re-fit on train and validation before evaluating on the held-out test set.

Models evaluated:

1. Naive Persistence Baseline

- $\log\text{-price}[t+7] = \log\text{-price}[t]$.
- Use `log_price_lag7` as prediction for `target_log_price` (assuming price persists over the horizon)

```
## Naive baseline (validation) RMSE (log): 0.09906 RMSE (price): 59.54 MAPE (%): 6.623
```

2. Ordinary Least Squares (OLS)

- Fit using selected lagged and rolling features: `log_price_lag1` ..`log_price_lag5`, `ma7`,`ma14`, and `vol7`.
- R^2 was 0.9944, which indicates suspicion that many variables could be highly correlated.
- Diagnostics revealed high multicollinearity, checked via VIF and Box test, making OLS not the most suitable model to represent prediction for this Bitcoin data.

```
## OLS validation RMSE (log): 0.07141 price-RMSE: 44.75 MAPE: 4.862
```

```
VIF Values:  ## log_price_lag1 log_price_lag2 log_price_lag3 log_price_lag4 log_price_lag5
##      3832.614163      3069.027825      2972.032802      3074.535199      3537.183053
##              ma7              ma14              vol7
##      37638.650949      1782.034754              1.281287
```

Box Test:

```
## Box-Ljung test
##
## data: resid_best
## X-squared = 3700.2, df = 20, p-value < 2.2e-16
```

3. Best-Subset Selection (using leaps)

- Best-Subset linear regression over lag and rolling features (allowing up to 10 predictors)
- Selected the subset size minimizing validation RMSE.

```
## # A tibble: 10 x 4
##   size RMSE RMSE_price MAPE
##   <int> <dbl> <dbl> <dbl>
## 1    10 0.0687    43.1  4.65
## 2     9 0.0687    43.1  4.66
## 3     8 0.0689    43.2  4.67
## 4     7 0.0695    43.3  4.68
## 5     6 0.0711    44.6  4.82
## 6     5 0.0721    45.1  4.91
## 7     4 0.0725    45.4  4.94
## 8     3 0.0743    46.5  5.09
## 9     2 0.0749    46.6  5.05
## 10    1 0.0751    46.7  4.98
```

```
## Best-subset (size= 10 ) test RMSE (log): 0.14524 price-RMSE: 1264.93 MAPE: 10.99
```

4. Lasso Regression

- Penalized regression and selected a sparse subset of predictors, improving interpretability.
- Non-zero coefficients mainly included shorter lags and the 7-day moving average.

```
## Best lambda (Lasso): 0.005994843
```

```
## Lasso (validation) - RMSE (log): 0.09549 RMSE (price): 61.95 MAPE (%): 8.133
```

Nonzero coefficients:

```
## [1] "(Intercept)"                "btc_total_bitcoins"
## [3] "btc_transaction_fees"        "btc_cost_per_transaction_percent"
## [5] "btc_cost_per_transaction"    "log_price_lag1"
## [7] "log_price_lag2"             "ma7"
## [9] "vol7"
```

5. Ridge Regression

- Penalized regression.
- Effective with multicollinear predictors.
- Distributed weight across correlated lags, but performed lagged behind Lasso.

```
## Best lambda (Ridge): 0.0002782559
```

Results

- **Naïve baseline (validation):**
 - RMSE (log) = **0.09906**
 - RMSE (price) = **59.54**
 - MAPE = **6.623%**.
- **OLS (hand-selected predictors) (validation):**
 - RMSE (log) = **0.07141**
 - RMSE (price) = **44.75**
 - MAPE = **4.862%**.
 - Note: OLS showed extremely large VIF values, indicating severe multicollinearity and invalidating coefficient interpretation and stability. This shows that using other models, such as lasso and ridge, might be more suitable for the data.
- **Best-subset (validation):**
 - Returned the **10-variable** subset as best by validation RMSE:
 - RMSE (log) = **0.0687**
 - RMSE (price) = **43.10**
 - MAPE = **4.65%**.

- **LASSO (validation):**
 - Best lambda = 0.005994843
 - RMSE (log) = **0.09549**
 - RMSE (price) = **61.95**
 - MAPE = **8.133%**.
- **Ridge (validation):**
 - Best lambda = 0.0002782559
 - RMSE (log) = **0.3783**
 - RMSE (price) = **195.52**
 - MAPE = **27.935%**

Final Test set evaluation (after refit on train and validation) : The final models were re-fit on train and validation and evaluated on the held-out test set.

Test metrics:	##	Model	RMSE_log	RMSE_price	MAPE
	##	<chr>	<dbl>	<dbl>	<dbl>
	## 1	Naive	0.214	1880.	16.5
	## 2	Best-Subset	0.145	1265.	11.0
	## 3	Ridge	0.318	2681.	18.9
	## 4	Lasso	0.225	3348.	18.5

Summary:

- OLS showed unstable coefficients and huge VIFs; despite good validation RMSE, its coefficient estimates are unreliable for interpretation or extrapolation.
- Ridge performed poorly in validation and test in this experiment, likely due to the particular penalty path and the fact that signals exist in a sparse subset of features
- LASSO performed better than ridge but worse than best-subset on the final test set

The best model on the test set is **Best-Subset linear regression with 10 predictors** because it had the lowest RMSE on log scale and price-scale RMSE (test RMSE log = **0.14524**, price RMSE $\approx$ **1264.93 USD**, MAPE $\approx$ **10.99%**). The best-subset outperformed regularization alternatives on the test block in this experiment.

